

TrafficAI — Model 2: Bus Lane Violation Detection

This document explains how the bus lane violation detection feature works end-to-end: the detection logic, the API contract (inputs and outputs), how to find the bus lane coordinates needed for testing, and how to test the feature in Postman.

1. Feature Overview

This feature answers one question for a given camera snapshot of a road with a marked bus lane: **which vehicles currently inside the bus lane are not allowed to be there?**

It works by combining two pieces of information:

- A **photo of the road** showing the bus lane (uploaded by the user/client).
- The **bus lane's boundary** — a set of pixel coordinates outlining the bus lane region in that exact photo (defined once per camera angle, using the coordinate-picking tool).

The system detects every vehicle in the photo using a YOLO object-detection model, then checks which of those detected vehicles fall inside the bus lane boundary. Any vehicle inside the lane that is **not a Bus** is flagged as an **unauthorized violation**. The result is returned as structured data, along with a copy of the original photo with the **bus lane outlined and a red box drawn around every violating vehicle**, so the result can be visually verified at a glance.

Why "Truck" is also treated as authorized

In Indian traffic, many buses — particularly mini-buses and those operated by private companies — are visually similar in shape and size to trucks. A general-purpose detection model can sometimes misclassify a real bus as a "Truck." To avoid incorrectly flagging genuine buses as violations because of this visual overlap, **both Bus and Truck are treated as authorized vehicles** for this feature. Every other detected vehicle type (Car, Bike, Bicycle, etc.) inside the lane is treated as a violation.

A known limitation

The detection model currently in use is a general-purpose model trained on a broad, global vehicle dataset. It recognizes common vehicle categories like **Car**, **Bike** (motorcycle), **Bicycle**, **Bus**, and **Truck** — but it does **not** have a dedicated category for **Auto-rickshaws**, which are common on Indian roads. Until the model is retrained specifically on Indian traffic data, an auto-rickshaw detected in the bus lane will most likely be labeled as the closest visual match the model can find — typically **Car** or **Bike** — rather than being labeled

"Auto-rickshaw" directly. This is a model-training limitation, not a logic issue, and can be resolved later by upgrading to a model trained specifically on Indian vehicle types.

2. How the Detection Logic Works

This is the core processing behind the bus lane violation endpoint. Its step-by-step process:

Step 1 — Decode the uploaded image correctly. Just like the parking feature, the photo is decoded in a way that **automatically corrects for camera rotation metadata**, ensuring the pixel layout matches exactly what a person sees when viewing the photo normally. This keeps the bus lane coordinates accurate regardless of how the photo was captured.

Step 2 — Build the bus lane boundary. The bus lane coordinates provided in the request (a set of points tracing the lane's outline) are converted into a boundary shape that the system can test other points against.

Step 3 — Run vehicle detection. The corrected image is passed through the detection model, which scans the entire photo and returns a list of every detected vehicle, each with a bounding box (its position and size in the photo), a vehicle type, and a confidence score (how sure the model is about that detection).

Step 4 — Reduce each vehicle to a single point. For every detected vehicle, the system calculates the **center point** of its bounding box. This center point is what gets checked against the bus lane boundary — a simpler and more reliable check than comparing two overlapping shapes.

Step 5 — Check which vehicles are inside the bus lane. For every detected vehicle, the system checks whether its center point falls inside the bus lane boundary. Vehicles whose center point falls **outside** the lane are ignored entirely — they're not relevant to this check.

Step 6 — Filter out authorized vehicles. Of the vehicles found inside the lane, any classified as **Bus** or **Truck** are considered authorized and excluded from the violation list. Everything else remaining is logged as a violation, recording its **vehicle type** and its **bounding box position** in the photo.

Step 7 — Calculate summary statistics. The system counts the total number of violations found, and calculates an average confidence score across just the flagged violating vehicles (not all vehicles in the photo) — giving a sense of how confident the system is in the violations it's reporting.

Step 8 — Draw the annotated image. A copy of the original (corrected) photo is annotated: the bus lane boundary is outlined, and each violating vehicle gets a red bounding box with a label showing its detected type (e.g. "Car", "Bike"). This gives a clear visual confirmation alongside the raw numbers.

Step 9 — Encode the annotated image. The annotated image is encoded into a Base64 text string (a way of representing image data as plain text), so it can be included directly inside a JSON API response rather than as a separate file.

Step 10 — Return the full result. The violation count, confidence score, the list of individual violations, and the Base64 annotated image are packaged together and returned as the API response.

3. API Endpoint: Bus Lane Violation Detection

Method: POST **Content type:** multipart/form-data (because an image file is being uploaded alongside text fields)

Input Fields

Field	Type	Required	Description
lane_image	File	Yes	The photo of the road/bus lane to analyze. Must be an image file.
signal_id	Number	Yes	An identifier for which signal/camera this photo belongs to (e.g. 12).
bus_lane_coordinates	Text (JSON string)	Yes	A JSON-encoded list of [x, y] pixel point pairs outlining the bus lane's boundary in this exact photo. 8 points is recommended (tracing both edges of the lane along its length), though any boundary with at least 3 points is accepted.

Important constraint: the **bus_lane_coordinates** must correspond to the *exact same image resolution and orientation* as the **lane_image** being uploaded. If the lane boundary was defined on a different photo, a resized photo, or a photo with different rotation, the results will be inaccurate or misaligned.

Output Fields

Field	Type	Description
unauthorized_count	Number	Total number of vehicles found inside the bus lane that are not authorized to be there.

Field	Type	Description
confidence_score	Number	Average detection confidence across the flagged violating vehicles only.
violations	List	One entry per violating vehicle — each has a type (the detected vehicle category, e.g. "Car", "Bike") and a bbox (its bounding box position in the photo, as [x1 , y1 , x2 , y2]).
annotated_image	Text (Base64)	The road photo with the bus lane boundary outlined and a red box drawn over every violating vehicle, encoded as a Base64 string. Can be decoded back into a viewable image.

Example Response

```
{ "unauthorized_count": 4, "confidence_score": 0.96, "violations": [ { "type": "Car",
"bbox": [240, 110, 340, 270] }, { "type": "Bike", "bbox": [520, 200, 590, 260] } ],
"annotated_image": "/9j/4AAQSkZJRgABAQAAQABAAAD... (long Base64 string)" }
```

Error Responses

Situation	Response
Uploaded file is not an image	400 Bad Request — "Uploaded file must be an image."
Uploaded image file is empty	400 Bad Request — "Empty image file."
bus_lane_coordinates isn't valid JSON	400 Bad Request — explains the expected JSON format.
bus_lane_coordinates has fewer than 3 points, or a point isn't a proper [x, y] pair	400 Bad Request — explains the coordinate format requirement.
The detection model file is missing on the server	500 Internal Server Error — points to the missing model path.

4. How to Find the Bus Lane Coordinates

Before testing the API, you need to define **where the bus lane is** in your test photo — these are the **bus_lane_coordinates** sent in every request. A dedicated coordinate-picking tool

exists for this purpose, separate from the parking slot picker (since this feature only needs one boundary per photo, not multiple).

What the tool does

The tool opens your chosen test photo in an interactive window. You **click points tracing the outline of the bus lane** directly on the image — typically 8 clicks: four points along the near (closer to camera) edge of the lane, and four along the far (further away) edge, to accurately capture the lane's shape as it narrows with perspective. Once you've placed your points, you save them, and the tool stores them into a single shared coordinates file — keyed by that image's filename — so you can build up coordinates for multiple test photos over time without overwriting earlier ones.

Why this matters for accuracy

The tool decodes the image using the **exact same orientation-correction logic** as the API itself, so whatever pixel position you click lines up precisely with how the API interprets that same photo. It also automatically scales large photos down to comfortably fit your screen for easier clicking, while still recording the true, full-resolution pixel coordinates underneath — so accuracy is preserved regardless of how zoomed-in or zoomed-out the on-screen view is.

Output of the tool

A shared JSON file where each entry is keyed by an image's filename, and its value is the list of [x, y] points outlining that image's bus lane, for example:

```
{ "Testing4.png": [[297, 177], [148, 414], [12, 771], [369, 820], [726, 817], [555, 361], [437, 184], [362, 171]] }
```

When testing a specific image, you only need that one image's coordinate list — the tool also prints it separately in a ready-to-paste format for convenience.

Tips for accurate clicking

- Click points **in order**, tracing around the lane boundary rather than jumping back and forth — this keeps the resulting shape a clean, non-crossing outline.
 - For a long, straight stretch of bus lane viewed at an angle, four points along each side (near edge and far edge) generally captures the perspective well.
 - Coordinates are tied to a specific camera position. If the same camera/angle is reused for live testing, the same coordinate set can be reused indefinitely — it only needs to be redone if the camera moves or the image resolution changes.
-

5. How to Test (Input Data & Process)

Step-by-step testing flow

1. **Choose a test photo** of a road with a visible bus lane (e.g. a file from your testing image set).
2. **Run the coordinate-picking tool** on that exact photo, clicking around the bus lane boundary (8 points recommended) and saving.
3. **Copy that image's coordinate list** from the tool's saved output.
4. **Send a test request** to the bus lane violation endpoint with:
 - The same photo as the **lane_image** input
 - A chosen **signal_id** (any identifying number for that camera/location)
 - The copied coordinate list as **bus_lane_coordinates**
5. **Review the response** — check the **unauthorized_count**, look through the **violations** list for the detected vehicle types and positions, and note the **confidence_score**.
6. **Decode and view the annotated_image** field to visually confirm the lane outline and red violation boxes match what's actually in the photo.

Sample test input (for reference)

Using a test image named **Testing4.png**, with a bus lane boundary already picked out as 8 points:

lane_image: Testing4.png (the photo file itself) **signal_id:** 12 **bus_lane_coordinates:** [[297, 177], [148, 414], [12, 771], [369, 820], [726, 817], [555, 361], [437, 184], [362, 171]]

This combination of an uploaded photo plus its matching coordinate set is exactly what the endpoint expects for a single test run.

What a good test result looks like

- The **annotated_image** shows the lane boundary closely tracing the actual painted bus lane in the photo.
- Every vehicle whose **center** is visibly within that lane gets a red box, **except** any genuine buses or trucks.
- The **violations** list's vehicle types roughly match what's visible (keeping in mind the auto-rickshaw labeling limitation noted earlier).
- If a vehicle is partially in the lane but its center point falls just outside the boundary, it correctly won't be flagged — this is expected behavior, not a bug, since the check is based on the vehicle's center position.

Things to double-check if results look wrong

- The **lane_image** uploaded for testing must be the **exact same file** used when picking coordinates — a different photo, a resized copy, or a re-exported version can shift everything out of alignment.
 - If violations seem to be missing for vehicles that are clearly inside the lane, check whether the vehicle's bounding box center genuinely falls inside your drawn boundary — this is most easily checked by viewing the annotated image.
 - If real buses are being flagged as violations, it likely means the model misclassified them as a vehicle type other than Bus or Truck — this would be a model-accuracy issue rather than a logic issue.
-

Note: Bus lane coordinates are tied to a specific camera position and image resolution, just like parking slot coordinates. If the camera is moved, repositioned, or the image resolution changes, the coordinate-picking step must be repeated for that new view.